Database Transactions – Lab Report

Prepared By: Kinza

December 7, 2025

Contents

1 Introduction to Database Transactions

1.1 Definition of a Transaction

A **transaction** in a database is a sequence of one or more SQL statements that perform a specific unit of work. Transactions ensure **data integrity** by allowing multiple operations to succeed or fail together, producing a consistent change in the database.

1.2 Components of a Database Transaction

- **DML Statements:** Represent changes to the data, such as `INSERT`, `UPDATE`, or `DELETE`.
- **DDL Statements:** Structural changes like `CREATE TABLE`, which automatically commit changes.
- **DCL Statements:** Control statements like `GRANT` or `REVOKE`, which also imply a commit.

1.3 Example of a Transaction

Consider transferring funds between two bank accounts:

- Debit $100 from account A
- Credit $100 to account B

If either operation fails, the transaction should fail entirely to maintain database consistency.

2 Transaction Control Commands

Oracle provides several commands to control transactions:

1. **COMMIT** - Makes all changes permanent in the database.
2. **ROLLBACK** - Reverts changes made since the start of the transaction or last savepoint.
3. **SAVEPOINT** - Sets a point in a transaction to which you can later rollback.
4. **SET TRANSACTION** - Configures transaction properties such as read/write mode.
5. **AUTOCOMMIT** - Automatically commits each DML statement.

2.1 Flow of a Transaction Example

```
-- Start a transaction
SET TRANSACTION NAME 'sal_update';

-- Update Banda's salary
UPDATE employees SET salary = 7000 WHERE last_name = 'Banda';

-- Set a savepoint
SAVEPOINT after_banda_sal;

-- Update Greene's salary
UPDATE employees SET salary = 12000 WHERE last_name = 'Greene';

-- Set another savepoint
SAVEPOINT after_greene_sal;

-- Rollback to Banda's savepoint
ROLLBACK TO SAVEPOINT after_banda_sal;

-- Update Greene's salary again
UPDATE employees SET salary = 11000 WHERE last_name = 'Greene';

-- Rollback entire transaction
ROLLBACK;

-- Start new transaction
SET TRANSACTION NAME 'sal_update2';
UPDATE employees SET salary = 7050 WHERE last_name = 'Banda';
UPDATE employees SET salary = 10950 WHERE last_name = 'Greene';

-- Commit changes
COMMIT;
```

3 Implicit and Explicit Transaction Control

3.1 Implicit Transaction Control

Oracle performs automatic commits or rollbacks in specific situations:

- **Automatic Commit:** When a DDL/DCL statement is executed or when exiting SQL*Plus normally.
- **Automatic Rollback:** Occurs on abnormal exits or system failures.

3.2 Explicit Transaction Control

Users manually control transactions using:

- `COMMIT`
- `ROLLBACK`
- `SAVEPOINT`

3.3 Advantages

- Ensures data consistency
- Allows preview of changes before committing
- Groups related operations for atomic execution

4 Creating and Modifying a Table in a Transaction

4.1 Creating a Table and Inserting Data

```
CREATE TABLE worker (
    worker_id NUMBER PRIMARY KEY,
    worker_name VARCHAR2(50),
    salary NUMBER
);

-- Insert a record
INSERT INTO worker (worker_id, worker_name, salary)
VALUES (1, 'Sohail', 5000);

-- Update salary without committing
UPDATE worker SET salary = 6000 WHERE worker_id = 1;
```

4.2 Handling Concurrent Transactions

If a second worksheet tries:

```
UPDATE worker SET salary = 7000 WHERE worker_id = 1;
```

It will wait until the first transaction is committed or rolled back.

4.3 Using SAVEPOINT and ROLLBACK

```
SET TRANSACTION NAME 'test_transaction';

-- Insert new worker
INSERT INTO worker VALUES (2, 'Erum', 5500);
SAVEPOINT sp1;

-- Update salary and set another savepoint
UPDATE worker SET salary = 6000 WHERE worker_name = 'Erum';
SAVEPOINT sp2;

-- Rollback to first savepoint
ROLLBACK TO SAVEPOINT sp1;

-- Commit transaction
COMMIT;
```

4.4 Using AUTOCOMMIT

```
SET AUTOCOMMIT ON;

INSERT INTO worker VALUES (3, 'FAST-NU', 5000);
-- This insert is committed automatically

SET AUTOCOMMIT OFF;
```

5 Scenario: Customer and Order Transactions

5.1 Tables Involved

- `customer`: Stores customer information
- `orders`: Stores orders related to customers

5.2 Transaction Steps

1. **Insert Customer:** Insert a new customer and create a savepoint.
2. **Insert Order and Deduct Balance:** Deduct the order amount and create another savepoint.
3. **Commit or Rollback:**
   - Commit if all operations succeed
   - Rollback to savepoint if an error occurs (like insufficient balance)

5.3 Example Query

```
SET TRANSACTION NAME 'customer_order_transaction';

-- Insert customer
INSERT INTO customer VALUES (101, 'Ali', 1000);
SAVEPOINT customer_added;

-- Insert order
INSERT INTO orders VALUES (201, 101, 200);
UPDATE customer SET balance = balance - 200 WHERE customer_id = 101;
SAVEPOINT order_added;

-- Commit if successful
COMMIT;
-- Rollback to customer_added if insufficient balance
ROLLBACK TO SAVEPOINT customer_added;
```

6 Practice Tasks for Students

6.1 Task 1: Product Inventory Transaction

```
CREATE TABLE product_inventory (
    product_id NUMBER PRIMARY KEY,
    product_name VARCHAR2(50),
    stock NUMBER,
    price NUMBER
);

CREATE TABLE returns (
    return_id NUMBER PRIMARY KEY,
    product_id NUMBER,
    return_date DATE,
    reason VARCHAR2(100),
    quantity NUMBER
);

-- Insert sample products
INSERT INTO product_inventory VALUES (1, 'Laptop', 10, 1500);
INSERT INTO product_inventory VALUES (2, 'Mouse', 50, 20);

-- Transaction with savepoint
SET TRANSACTION NAME 'inventory_update';

INSERT INTO returns VALUES (1, 1, SYSDATE, 'Defective', 2);
SAVEPOINT stock_update;
```

```
25
26 -- Update inventory
27 UPDATE product_inventory
28 SET stock = stock + 2 * 0.95
29 WHERE product_id = 1;
30
31 -- Rollback if required
32 ROLLBACK TO SAVEPOINT stock_update;
```

6.2 Task 2: Bank Accounts Transaction with Fee

```
1  CREATE TABLE bank_accounts (
2      account_id NUMBER PRIMARY KEY,
3      balance NUMBER,
4      account_type VARCHAR2(20)
5  );
6
7  CREATE TABLE transactions (
8      transaction_id NUMBER PRIMARY KEY,
9      account_id NUMBER,
10     type VARCHAR2(10),
11     amount NUMBER,
12     transaction_date DATE
13 );
14
15 -- Transfer with 2% fee
16 SET TRANSACTION NAME 'fund_transfer';
17
18 UPDATE bank_accounts
19 SET balance = balance - 102 -- 100 + 2% fee
20 WHERE account_id = 1;
21 SAVEPOINT balance_updated;
22
23 UPDATE bank_accounts
24 SET balance = balance + 100
25 WHERE account_id = 2;
26
27 -- Rollback if balance insufficient
28 ROLLBACK TO SAVEPOINT balance_updated;
```

6.3 Task 3: Loyalty Points Transaction

```
1  CREATE TABLE customers (
2      customer_id NUMBER PRIMARY KEY,
3      name VARCHAR2(50)
```

```
);

CREATE TABLE loyalty_program (
    customer_id NUMBER,
    points_earned NUMBER,
    points_redeemed NUMBER
);

-- Transaction to award points
SET TRANSACTION NAME 'loyalty_update';

UPDATE loyalty_program
SET points_earned = points_earned + (CASE
    WHEN purchase_amount > 500 THEN purchase_amount * 1.5
    ELSE purchase_amount * 1
END);

SAVEPOINT points_updated;

-- Rollback if points calculation issue
ROLLBACK TO SAVEPOINT points_updated;

COMMIT;
```

7 Conclusion

Database transactions allow grouping multiple SQL operations into a single unit of work. Using **COMMIT**, **ROLLBACK**, and **SAVEPOINT**, users can maintain data consistency, preview changes, and recover from errors. Transactions are crucial in banking, inventory management, and order processing to prevent inconsistent or partial data updates.